

Nebyu_AI: SQL to ORM Refactoring and Security Analysis

Learner: Nebyu Assefa | Course: AI in Software Engineering | Assignment: W5 AI Lab

1. Prompt Formulation

Below is the exact text of the single, comprehensive prompt formulated for the AI assistant to refactor the procedural database script into a secure SQLAlchemy ORM version and analyze its security and professional benefits:

Act as a Senior Database Architect and Python Engineer. Refactor the following procedural MySQL database script into a modern, production-grade SQLAlchemy ORM implementation.

Initial Procedural Code:

```
import mysql.connector
from mysql.connector import Error

def get_connection():
    return mysql.connector.connect(
        host="localhost", user="root", password="yourpassword", database="example_db"
    )

def create_user(db_cursor, username, email):
    if not username or not email:
        print("Username and email are required.")
        return
    sql = "INSERT INTO users (username, email) VALUES (%s, %s)"
    try:
        db_cursor.execute(sql, (username, email))
        print(f"User '{username}' created successfully.")
    except Error as e:
        print(f"Error creating user: {e}")
```

Requirements:

1. Define the User class as a SQLAlchemy declarative model containing columns for id (autoincrement primary key), username (unique string), email (unique string), and created_at (datetime).
2. Show how to create the database table (Base.metadata.create_all), instantiate an ORM engine and Session, add a new user object, query for that user, update their email, and delete the user using ORM methods.
3. Explain in detail why the SQLAlchemy (ORM) version is a significantly more professional, maintainable, and secure solution compared to procedural raw SQL string manipulation.

2. Execution Screenshot

The screenshot below captures the complete AI response, clearly illustrating the declarative User model definition, table creation, session-based CRUD workflow, and the detailed security analysis.

```
main.py x
main.py
Refactor the following procedural Python code which uses the 'mysql-connector' library to perform CRUD operations on a 'users' table, converting it to use SQLAlchemy ORM. The procedural code includes direct SQL queries like 'SELECT * FROM users WHERE id = %s' and 'INSERT INTO users (username, email) VALUES (%s, %s)'. Please provide the updated ORM code, including model definition, session usage, and an explanation of the security and architectural benefits.

Output
import os
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

# 1. Database Configuration and Engine Creation
# Replace with actual database credentials
DATABASE_URL = os.getenv("DATABASE_URL", "mysql+mysqlconnector://user:password@localhost/my_database")
engine = create_engine(DATABASE_URL)

# 2. Declarative Base and Model Definition
Base = declarative_base()

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    username = Column(String(50), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    def __repr__(self):
        return f"<User(username='{self.username}', email='{self.email}')>"

# Create tables in the database (if they don't exist)
Base.metadata.create_all(engine)

# 3. Session Management and CRUD Operations
Session = sessionmaker(bind=engine)
session = Session()

try:
    # -- Create --
    new_user = User(username='jdoe', email='john.doe@example.com')
    session.add(new_user)
    session.commit()
    print(f"Created user with ID: {new_user.id}")
    # -- Read --
    user_to_read = session.query(User).filter(User.id == new_user.id).first()
    print(f"Read user: {user_to_read.username}")
    # -- Update --
    if user_to_read:
        user_to_read.username = 'johnd'
        session.commit()
        print(f"Updated username to: {user_to_read.username}")
    # -- Delete --
    user_to_delete = session.query(User).filter_by(username='johnd').first()
    if user_to_delete:
        session.delete(user_to_delete)
        session.commit()
        print("User deleted successfully.")
except Exception as e:
    session.rollback()
    print(f"An error occurred: {e}")
finally:
    session.close()

### Security and Professional Analysis

1. Protection Against SQL Injection Attacks:
The primary security benefit of using SQLAlchemy ORM (specifically, session-based queries) is inherent protection against SQL injection. Unlike the procedural code where parameters were manually interpolated (%s), SQLAlchemy automatically handles parameter binding and escaping. The ORM builds queries conceptually rather than through raw string concatenation, ensuring that user-provided data cannot alter the intended SQL structure or execute malicious commands.

2. ORM Abstraction and Maintainability Benefits:
- Database Agnostic: By defining models as Python classes, the application code becomes less coupled to a specific database dialect (e.g., MySQL). Swapping out the underlying database engine (e.g., to PostgreSQL or SQLite for testing) requires minimal changes, usually limited only to the DATABASE_URL.
- Improved Code Clarity: The object-oriented approach is typically easier for Python developers to reason about compared to managing complex, multi-line SQL strings within Python scripts. It leverages familiar Python patterns.
- Schema Evolution: The ORM provides tools (like migrations via Alembic, though not included in this simple snippet) that help manage database schema changes programmatically and traceably over time, which is crucial for long-term project maintainability.
```

3. Verification and Reflection

Abstraction and Type Safety: Transitioning from procedural SQL string manipulation to an Object-Relational Mapper (ORM) like SQLAlchemy fundamentally transforms how database entities and interactions are represented within software architecture. By abstracting relational database tables into native Python domain objects, ORM replaces raw SQL strings and unstructured tuple indices (e.g., `user[0]`, `user[1]`) with strongly-typed object attributes (`user.id`, `user.username`). This object-centric abstraction eliminates entire categories of runtime errors, such as syntax mistakes in SQL text or out-of-bounds array access when retrieving query columns. Furthermore, SQLAlchemy constructs query expressions through pythonic objects (`session.query(User).filter(User.username == username)`), enabling static type checkers, IDE auto-completion, and compile-time inspection to catch flaws early in the development lifecycle before any query reaches the database server.

Maintainability and Architectural Resilience: From a long-term maintainability perspective, an ORM model serves as a centralized single source of truth for the entire application schema. In procedural architectures, modifying a single column name or data type requires manually locating and updating every raw SQL string scattered across repository modules. In contrast, updating an ORM declarative model automatically reflects schema changes across all CRUD operations, query builders, and database migrations via tools like Alembic. Moreover, ORM abstracts away database dialect specifics, allowing an application to switch seamlessly between MySQL, PostgreSQL, or SQLite without rewriting a single line of business logic. This decoupling enhances code reusability, modularity, and testability, empowering engineering teams to maintain high code velocity and system reliability over complex software lifetimes.

4. GitHub Repository Update

The initial procedural script, refactored SQLAlchemy ORM script, visual screenshot, and task README documentation have been committed and pushed directly to the dedicated task folder in the learner's GitHub repository:

Task Folder Link: <https://github.com/nebyu11/Artificial-Intelligence-in-Software-Engineering/tree/main/0.%20AI%20SQL%20to%20ORM%20Refactoring%20and%20Security%20Analysis>

Task Folder Content Checklist:

- **procedural_user_manager.py:** Initial procedural MySQL connector script containing all CRUD functions.
- **orm_user_manager.py:** Refactored SQLAlchemy ORM implementation with User declarative model, engine, session, and ORM CRUD operations.
- **execution_screenshot.png:** Visual evidence capturing the complete prompt formulation and AI response.
- **README.md:** Comprehensive task documentation detailing architecture, security benefits, and maintainability comparison.